

Modelo físico e integridade no MySQL — T05

Larissa de Lima Oliveira – Ciências da Computação - 825130676 - Butantã/Noite

Exercícios de Fixação — Respostas

NÍVEL 1 — RECONHECIMENTO

Escolha tipos para um sistema acadêmico.

- Modele ALUNO (id, matrícula, nome, e-mail, nascimento, coeficiente, bolsista e observações).
- Escolha um tipo MySQL para cada atributo.
- Defina comprimentos e precisão com justificativas.

Atributo	Tipo	Justificativa
id	INT NOT NULL AUTO_INCREMENT	Identificador substituto, gerado pelo sistema
matrícula	CHAR (10)	Código de tamanho fixo
Nome	VARCHAR (100)	Texto curto
e-mail	VARCHAR (254)	Identificador alternativo, limite de e-mail
nascimento	DATE	Data civil
coeficiente	DECIMAL (5,2)	Precisão decimal exata
bolsista	BOOLEAN	Estado lógico verdadeiro/falso
observação	TEXT	Conteúdo livre

- Identifique códigos que não devem ser tratados como números.

Matrícula mesmo sendo dígitos é identificador, não quantidade é a mesma coisa de um CPF.

- Diferencie dado ausente, inaplicável e obrigatório.

Dado ausente: o fato existe no mundo real, mas ainda não foi registrado no banco, é uma lacuna temporária de informação.

Dado inaplicável: o atributo não se aplica àquele registro, o fato simplesmente não existe para ele, então não há nada a preencher.

Dado obrigatório: o banco não funciona sem esses dados.

NÍVEL 2 — APLICAÇÃO

Declare a tabela RESERVA com constraints coerentes.

- Use id_reserva, id_sala, id_responsavel, inicio, fim e situacao.
- Defina PK, FKs, NOT NULL e DEFAULT.
- Imponha CHECK para fim posterior a inicio.
- Controle o vocabulário de situacao.

```
CREATE TABLE reserva (  
  id_reserva INT PRIMARY KEY auto_increment,  
  id_sala INT NOT NULL,  
  id_responsavel INT NOT NULL,  
  inicio DATETIME NOT NULL,  
  fim DATETIME NOT NULL,  
  situacao VARCHAR (20) NOT NULL DEFAULT 'CONFIRMADA',  
  CONSTRAINT pk_reserva PRIMARY KEY (id_reserva),  
  CONSTRAINT ck_reserva_periodo CHECK (fim > inicio),  
  CONSTRAINT ck_situacao CHECK (situacao IN ('CONFIRMADA', 'CANCELADA',  
'PENDENTE')),  
  CONSTRAINT fk_reserva_sala FOREIGN KEY (id_sala) REFERENCES sala(id_sala)  
  ON DELETE RESTRICT ON UPDATE RESTRICT,  
  CONSTRAINT fk_reserva_responsavel FOREIGN KEY (id_responsavel) REFERENCES  
responsavel(id_responsavel)  
  ON DELETE RESTRICT ON UPDATE RESTRICT,  
) ENGINE = InnoDB;
```

- Explique por que UNIQUE (id_sala, inicio) não detecta toda sobreposição temporal.
UNIQUE (id_sala, inicio) só impede reservas com o mesmo horário de início na mesma sala. Isso acontece porque UNIQUE só compara valores entre linhas.

NÍVEL 3 — DECISÃO

Escolha ON DELETE e ON UPDATE para três vínculos.

- DEPARTAMENTO → EMPREGADO: o empregado pode permanecer sem departamento?
- PROJETO → TAREFA: a tarefa possui existência independente?
- AUTOR → LIVRO_AUTOR: o contrato histórico pode ser apagado?
- Para cada vínculo, escolha RESTRICT, CASCADE ou SET NULL.
- Justifique pela obrigatoriedade, história e ciclo de vida — não por conveniência.

Vínculos	Pergunta - chave	Ação	Justificativa
DEPARTAMENTO → EMPREGADO	o empregado pode permanecer sem departamento?	ON DELETE SET NULL	o empregado pode continuar existindo sem departamento até ser realocado
PROJETO → TAREFA	a tarefa possui existência independente?	ON DELETE CASCADE	Tarefa é parte composicional do projeto
AUTOR → LIVRO_AUTOR	o contrato histórico pode ser apagado?	ON DELETE RESTRICT	É registro histórico de autoria; excluir o autor não pode apagar esse vínculo

NÍVEL 4 — TESTE

Projete violações que demonstrem a eficácia das constraints

- Crie um INSERT que viole PRIMARY KEY.

```
Viola PRIMARY KEY (id repetido)
INSERT INTO cliente (id_cliente, nome, email) VALUES (1, 'Ana',
'ana@x.com');
INSERT INTO cliente (id_cliente, nome, email) VALUES (1, 'Beatriz',
'bia@x.com');
-- ERRO: Duplicate entry '1' for key 'PRIMARY'
```

- Crie um INSERT que viole UNIQUE e outro que viole CHECK.

```
Viola UNIQUE (email duplicado)
INSERT INTO cliente (nome, email) VALUES ('Carlos', 'ana@x.com');
-- ERRO: Duplicate entry 'ana@x.com' for key 'uq_cliente_email'
Viola CHECK (preço negativo)
INSERT INTO produto (nome, preco_atual) VALUES ('Caneta', -5.00);
-- ERRO: Check constraint 'ck_produto_preco' is violated
```

- Tente inserir uma FK órfã.

```
FK órfã (cliente inexistente)
INSERT INTO pedido (id_cliente) VALUES (9999);
-- ERRO: Cannot add or update a child row: a foreign key constraint fails
```

- Teste uma exclusão bloqueada por RESTRICT.

```
Exclusão bloqueada por RESTRICT
DELETE FROM cliente WHERE id_cliente = 1;
-- ERRO se houver pedidos vinculados a esse cliente
```

- Teste uma exclusão propagada por CASCADE e descreva o estado final esperado.

```
Exclusão propagada por CASCADE
DELETE FROM pedido WHERE id_pedido = 10;
-- Sucesso: a linha do pedido 10 é removida, e o MySQL automaticamente
-- remove também todas as linhas de item_pedido com id_pedido = 10
```

NÍVEL 5 — ANÁLISE

Encontre redundâncias e anomalias em uma relação não normalizada.

- Análise VENDA (id_venda, data, id_cliente, nome_cliente, id_produto, nome_produto, quantidade).
- Identifique os fatos distintos misturados.

Dado do cliente - Só id_cliente já pode identificar os dados deles, não a necessidade do nome_cliente.

Dado do produto - Só id_produto já pode identificar os dados deles, não a necessidade do nome_produto.

- Demonstre uma anomalia de atualização, uma de inserção e uma de exclusão.

Atualização: se o cliente trocar de nome, é preciso atualizar nome_cliente em todas as vendas dele

Inserção: não dá para cadastrar um produto novo sem que ele já tenha uma venda associada

Exclusão: apagar a única venda de um cliente apaga também o único lugar onde o nome dele estava registrado.

- Decomponha a relação em tabelas com PKs e FKs.

```
CREATE TABLE cliente (  
  id_cliente INT NOT NULL AUTO_INCREMENT,  
  nome_cliente VARCHAR(100) NOT NULL,  
  PRIMARY KEY (id_cliente)  
) ENGINE = InnoDB;  
  
CREATE TABLE produto (  
  id_produto INT NOT NULL AUTO_INCREMENT,  
  nome_produto VARCHAR(160) NOT NULL,  
  PRIMARY KEY (id_produto)  
) ENGINE = InnoDB;  
  
CREATE TABLE venda (  
  id_venda INT NOT NULL AUTO_INCREMENT,  
  data DATE NOT NULL,  
  id_cliente INT NOT NULL,  
  id_produto INT NOT NULL,  
  quantidade INT NOT NULL,  
  preco DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_venda PRIMARY KEY (id_venda),
```

```
CONSTRAINT ck_qtd CHECK (quantidade > 0),
CONSTRAINT fk_venda_cliente FOREIGN KEY (id_cliente) REFERENCES
cliente(id_cliente)
ON DELETE RESTRICT,
CONSTRAINT fk_venda_produto FOREIGN KEY (id_produto) REFERENCES
produto(id_produto)
ON DELETE RESTRICT
) ENGINE = InnoDB;
```

- Indique qual atributo histórico deve permanecer no item da venda.

O preço do produto no catálogo pode mudar depois; a venda precisa guardar o valor congelado no momento da transação, senão vendas antigas passariam a refletir em preços futuros.

DESAFIO FINAL

Implemente o modelo físico de uma clínica.

- Crie schema e tabelas PACIENTE, MÉDICO, CONSULTA, PROCEDIMENTO e CONSULTA_PROCEDIMENTO.

```
CREATE DATABASE IF NOT EXISTS clinica
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;
USE clinica;

CREATE TABLE paciente (
  id_paciente      INT NOT NULL AUTO_INCREMENT,
  nome             VARCHAR(100) NOT NULL,
  cpf             CHAR(11) NOT NULL,
  data_nascimento  DATE NOT NULL,
  telefone        VARCHAR(20),
  CONSTRAINT pk_paciente PRIMARY KEY (id_paciente),
  CONSTRAINT uq_paciente_cpf UNIQUE (cpf)
) ENGINE = InnoDB;

CREATE TABLE medico (
  id_medico        INT NOT NULL AUTO_INCREMENT,
  nome             VARCHAR(120) NOT NULL,
  crm              VARCHAR(20) NOT NULL,
  especialidade    VARCHAR(50) NOT NULL,
  CONSTRAINT pk_medico PRIMARY KEY (id_medico),
  CONSTRAINT uq_medico_crm UNIQUE (crm)
) ENGINE = InnoDB;

CREATE TABLE procedimento (
  id_procedimento  INT NOT NULL AUTO_INCREMENT,
  nome             VARCHAR(120) NOT NULL,
  valor           DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_procedimento PRIMARY KEY (id_procedimento),
  CONSTRAINT ck_procedimento_valor CHECK (valor >= 0)
) ENGINE = InnoDB;

CREATE TABLE consulta (
  id_consulta      INT NOT NULL AUTO_INCREMENT,
  id_paciente      INT NOT NULL,
  id_medico        INT NOT NULL,
  data_hora        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status           VARCHAR(20) NOT NULL DEFAULT 'AGENDADA',
  CONSTRAINT pk_consulta PRIMARY KEY (id_consulta),
  CONSTRAINT ck_consulta_status CHECK (status IN
('AGENDADA', 'REALIZADA', 'CANCELADA')),
```

```

        CONSTRAINT fk_consulta_paciente FOREIGN KEY (id_paciente) REFERENCES
paciente(id_paciente)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
        CONSTRAINT fk_consulta_medico FOREIGN KEY (id_medico) REFERENCES
medico(id_medico)
        ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;

CREATE TABLE consulta_procedimento (
    id_consulta      INT NOT NULL,
    id_procedimento  INT NOT NULL,
    valor_praticado  DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_consulta_procedimento PRIMARY KEY (id_consulta,
id_procedimento),
    CONSTRAINT ck_cp_valor CHECK (valor_praticado >= 0),
    CONSTRAINT fk_cp_consulta FOREIGN KEY (id_consulta) REFERENCES
consulta(id_consulta)
        ON DELETE CASCADE,
    CONSTRAINT fk_cp_procedimento FOREIGN KEY (id_procedimento) REFERENCES
procedimento(id_procedimento)
        ON DELETE RESTRICT
) ENGINE = InnoDB;

```

- Escolha tipos para identificadores, textos, data_hora, valores e estados.

Coluna	Tipo	Justificativa
cpf	CHAR (11)	Tamanho fixo
crm	VARCHAR (20)	Formato padrão
Data_hora	DATETIME	Horário da consulta importa
valor / valor_praticado	DECIMAL (10,2)	Precisão financeira exata
status	VARCHAR (20) + CHECK	Vocabulário controlado

- Declare PKs, FKs, UNIQUE, NOT NULL, DEFAULT e CHECK.

Constraint	Onde aparece
PKs	id_paciente; id_medico; id_procedimento; id consulta; consulta_procedimento
FKs	consulta_paciente; consulta_medico; cp_consulta; cp_procedimento
UNIQUE	paciente_cpf; medico_crm
NOT NULL	Presente em praticamente todas as colunas obrigatórias de todas as tabelas

DEFAULT	consulta.data_hora; consulta.status
CHECK	procedimento.valor_padrao; consulta.status; consulta_procedimento.valor_praticado

- Defina e justifique ON DELETE e ON UPDATE para cada vínculo.

<i>Vínculo</i>	<i>Ação</i>	<i>Justificativa</i>
PACIENTE → CONSULTA	RESTRICT	Consulta é histórico clínico; não pode sumir se o paciente for excluído
MÉDICO → CONSULTA	RESTRICT	histórico de atendimento não deve desaparecer
CONSULTA → CONSULTA_PROCEDIMENTO	CASCADE	Procedimento realizado não existe sem a consulta
PROCEDIMENTO → CONSULTA_PROCEDIMENTO	RESTRICT	O procedimento é catálogo reutilizável; não pode ser excluído se já foi usado em atendimentos

- Planeje ao menos seis testes de violação e preveja cada resultado.

```
-- 1) CPF duplicado
INSERT INTO paciente (nome, cpf, data_nascimento) VALUES ('Ana',
'11122233344', '1990-01-01');
INSERT INTO paciente (nome, cpf, data_nascimento) VALUES ('Bia',
'11122233344', '1992-05-10');
-- Esperado: rejeitado (UNIQUE uq_paciente_cpf)

-- 2) Valor negativo em consulta_procedimento
INSERT INTO consulta_procedimento (id_consulta, id_procedimento,
valor_praticado)
VALUES (1, 1, -50.00);
-- Esperado: rejeitado (CHECK ck_cp_valor)

-- 3) Consulta com médico inexistente
INSERT INTO consulta (id_paciente, id_medico) VALUES (1, 9999);
-- Esperado: rejeitado (FOREIGN KEY fk_consulta_medico)

-- 4) Excluir paciente com consultas
DELETE FROM paciente WHERE id_paciente = 1;
-- Esperado: rejeitado (ON DELETE RESTRICT)

-- 5) Excluir consulta com procedimentos associados
DELETE FROM consulta WHERE id_consulta = 1;
-- Esperado: aceito; linhas de consulta_procedimento com id_consulta = 1
-- são removidas automaticamente (CASCADE)
```

```
-- 6) Status fora do vocabulário
INSERT INTO consulta (id_paciente, id_medico, status) VALUES (1, 1,
'PENDENTE');
-- Esperado: rejeitado (CHECK ck_consulta_status)
```

- Revise redundâncias, dados históricos e possíveis anomalias.

Sem redundância problemática: não duplica nome do médico, do paciente nem do procedimento em nenhum lugar.

Anomalia evitada: como paciente, médico e procedimento são entidades independentes com suas próprias tabelas, excluir a última consulta de um médico não apaga o cadastro do médico.

EXERCÍCIO • AUTOAVALIAÇÃO

- Por que CPF deve ser CHAR/VARCHAR e preço deve ser DECIMAL?
 - o CPF é um identificador, não quantidade, já preço participa de operações e exige precisão exata.
- Qual diferença existe entre PRIMARY KEY, UNIQUE e NOT NULL?
 - o PRIMARY KEY: identidade única e obrigatoriamente não nula.
 - o UNIQUE: impede valores repetidos numa coluna.
 - o NOT NULL: só garante que a coluna sempre tenha um valor.
- Quando ON DELETE CASCADE é coerente — e quando é perigoso?
 - o É coerente quando o filho é parte composicional do pai, sem existência própria. É perigoso quando usado sem analisar o ciclo de vida real podendo causar exclusões em massa não intencionais, especialmente se propagar por várias tabelas encadeadas.
- Por que SET NULL exige uma FK anulável?
 - o Porque SET NULL funciona removendo o valor da FK (colocando NULL) quando o pai é excluído.
- Como uma redundância produz anomalia de atualização?
 - o Quando o mesmo fato é guardado em múltiplos lugares independentes, uma mudança nesse fato exige atualizar todas as cópias.
- Que violações você testaria antes de considerar o schema confiável?
 - o No mínimo: PK duplicada, UNIQUE duplicada, CHECK violado, exclusão de um pai com filhos sob RESTRICT (deve bloquear), exclusão de um pai com filhos sob CASCADE (deve propagar corretamente), cobrindo os três níveis de integridade (domínio, entidade, referencial) e confirmando que cada ação referencial se comporta como projetado.